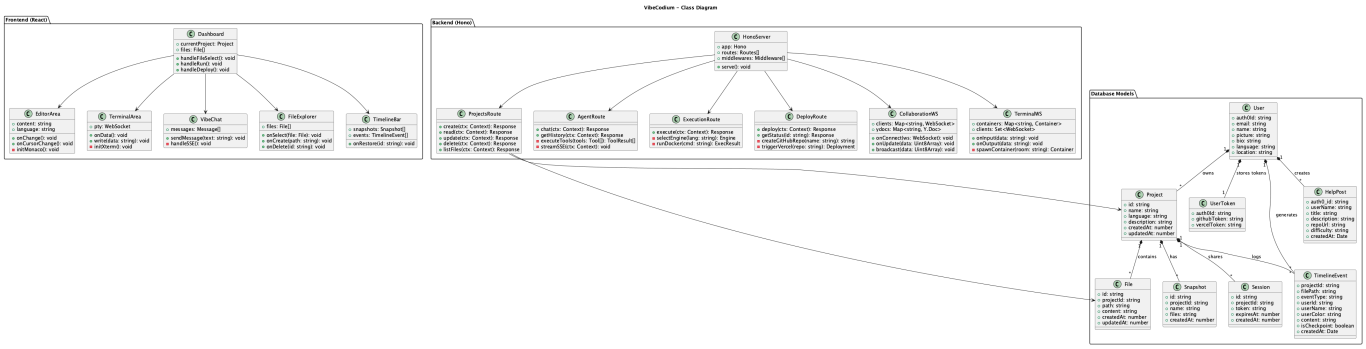
VibeCodium - Architecture & Diagrams Explained

VibeCodium is a collaborative real-time code editor and isolated execution sandbox designed to streamline development.

Tech Stack:

- **Frontend**: React 19, Vite, Monaco Editor, Xterm.js
- **Backend**: Hono
- **Real-Time Collaboration**: Yjs (CRDT) over WebSockets
- **Databases**: Supabase (Cloud Storage)
- **Code Execution**: Docker Sandboxes
- **Deployment**: Vercel

1. The Class Diagram

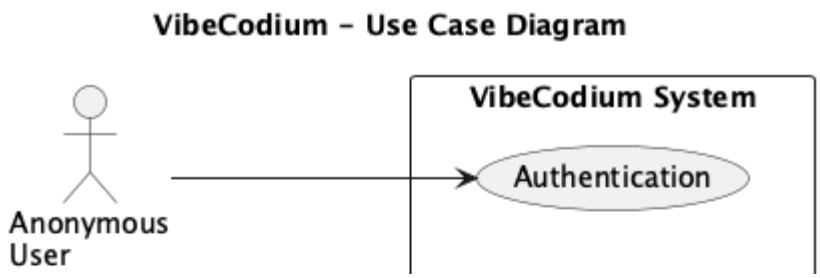


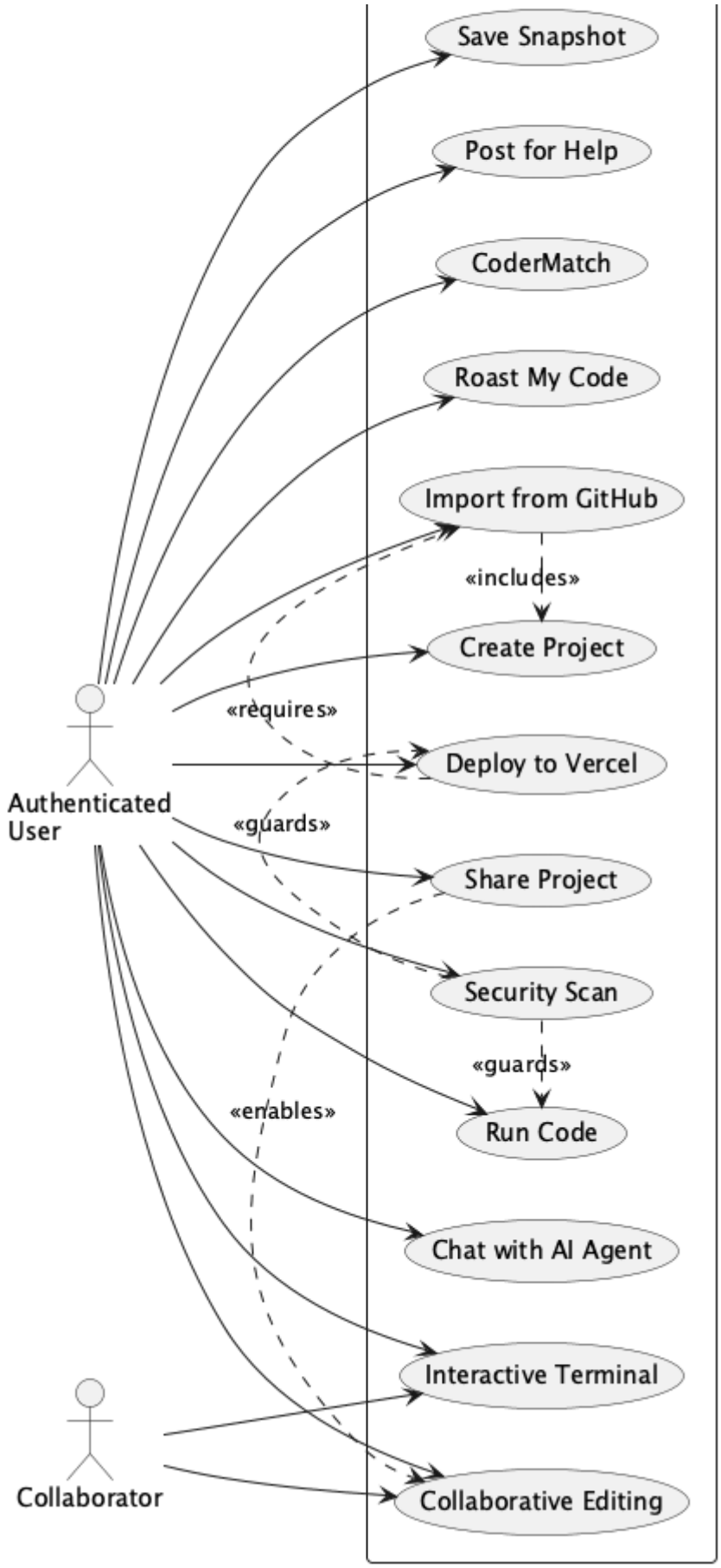
The Class Diagram is essentially the blueprint of our entire codebase. It breaks the system down into three main pieces:

- **Frontend (React)**: Everything revolves around the `Dashboard`, which acts as the orchestrator for the `EditorArea` (where the magic happens with Monaco), the `TerminalArea` (hooked up to Xterm.js for that sweet interactive PTY terminal), and our AI assistant inside `VibeChat`.
- **Backend (Hono)**: This is our lightweight, ultra-fast API. Aside from handling the standard REST routes (like managing projects or executing code sandbox environments), it spins up our WebSocket handlers for real-time multiplayer editing (`CollaborationWS`) and terminal streaming (`TerminalWS`).
- **Database Models**: This section highlights the core entities that store everything from the `User` and `Project` details down to timeline `Snapshots` and `HelpPost` interactions.

---

2. The Use Case Diagram





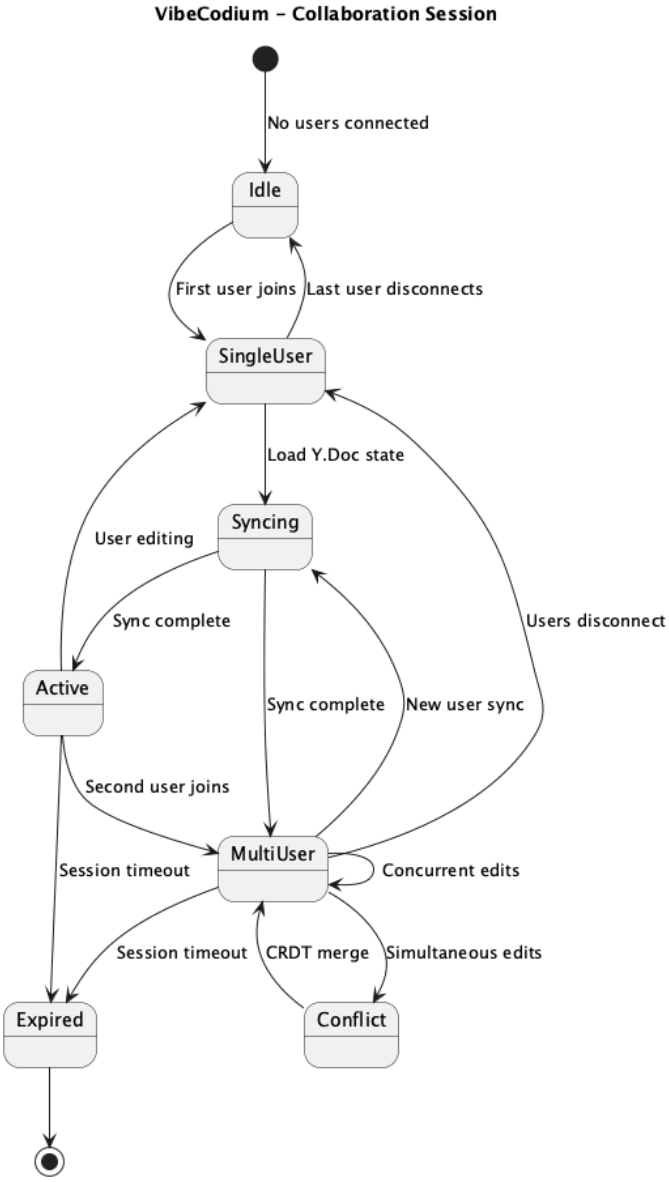
Anonymous Users: They only view the initial landing experience to authenticate via Auth0.

Authenticated Users: This is where all the action happens. Once logged in, you can create projects, spin up code in the Docker sandboxes, deploy instantly to Vercel, chat with the integrated AI, or browse community features like CoderMatch and "Roast My Code."

Collaborators: They can enter a project via a shared token link. They get dropped right into the action, gaining access to the shared collaborative editor and the interactive terminal.

---

3. The State Diagram (Collaboration Session)



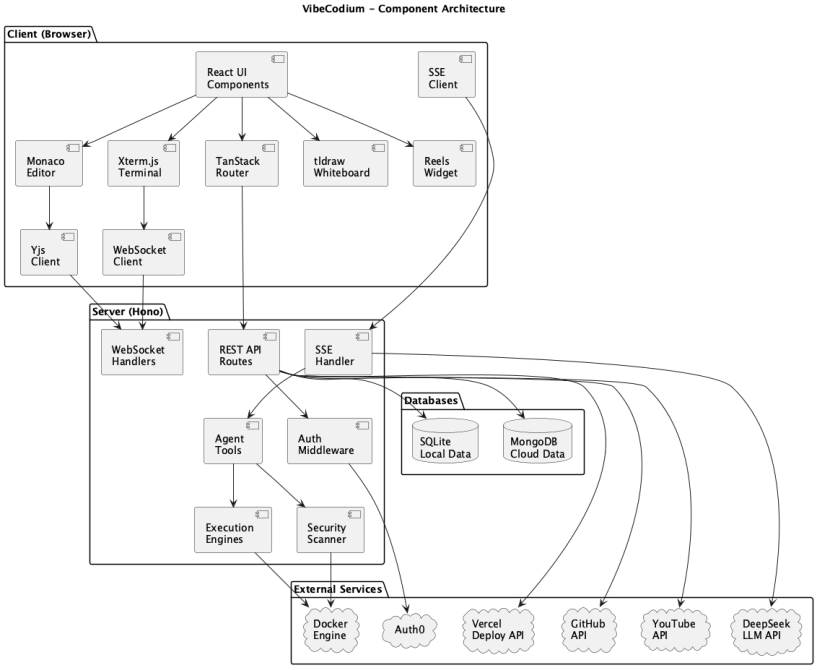
This diagram gets into the real-time collaboration. It shows exactly what happens under the hood when multiple people are typing code simultaneously.

Here is the lifecycle of a multiplayer session:

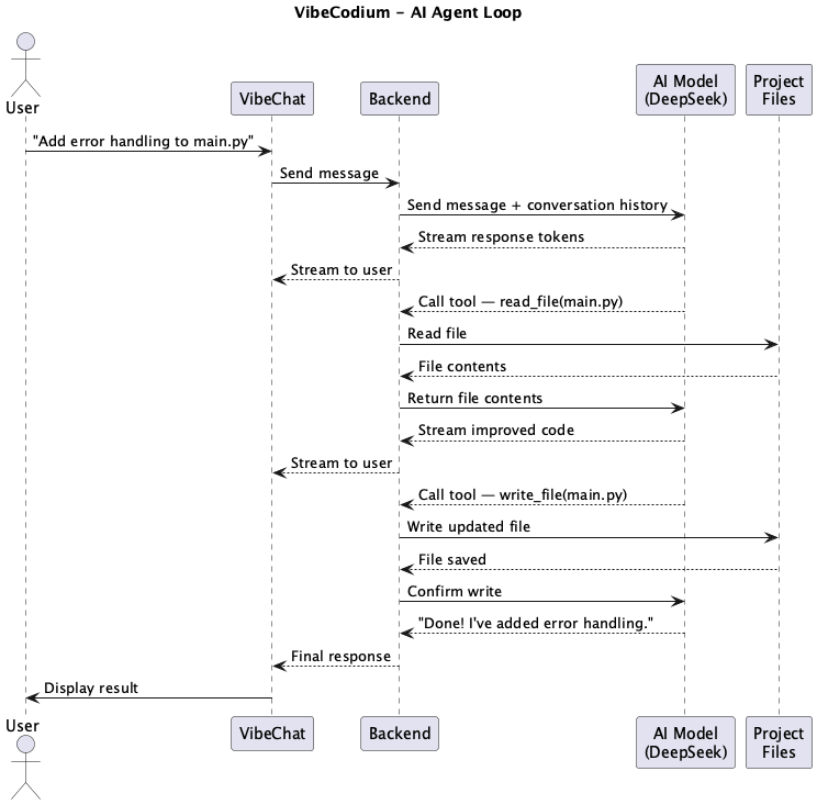
- It starts off `Idle` until the first person connects, flipping the state to `SingleUser`.
- Whenever someone new joins the party, the system briefly enters a `Syncing` state. During this tiny window, the client syncs up with the existing `Y.Doc` state stored on the server.
- Once multiple people are active (`MultiUser`), we rely on CRDT (Conflict-free Replicated Data Type) magic. If two people aggressively edit the exact same lines at the same time, the system seamlessly triggers a CRDT merge. No locked files, no annoying conflict pop-ups.
- Eventually, if everyone leaves and the session reaches its max inactivity limit, it transitions safely to an `Expired` state.

---

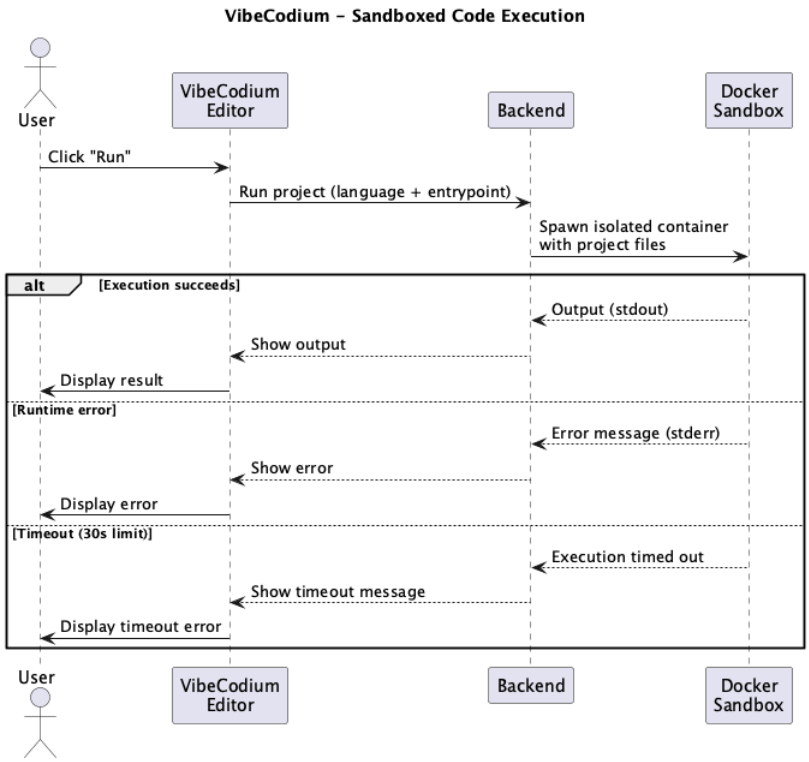
4. Component Diagram



5. Sequence Diagram: AI Agent Loop

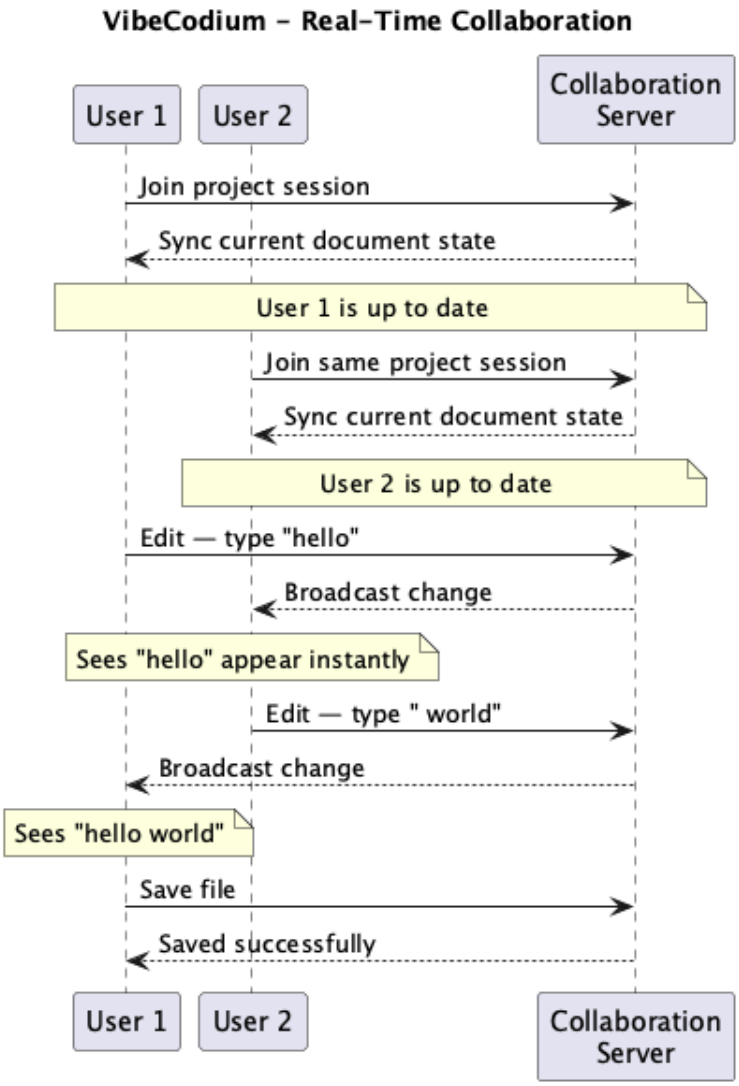
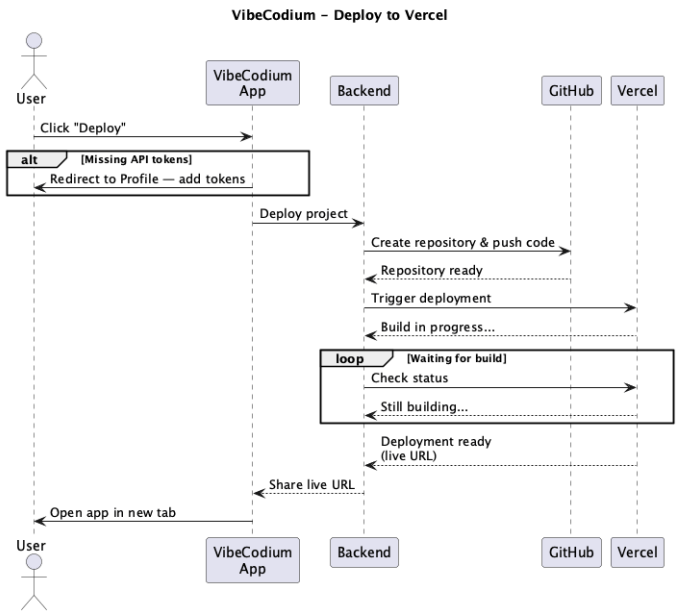


6. Sequence Diagram: Docker Execution



---

7. Sequence Diagram: Real-Time Collaboration



8. Sequence Diagram: Vercel Deploy